6. Matrix Multiplication

```
import numpy as np
def matrix_multiplication(A, B):
    if A.shape[1] != B.shape[0]:
        raise ValueError("Number of columns in A must be equal to number of rows in B")
    result = np.dot(A, B)
    return result
if __name__ == "__main__":
    # Define two matrices
    A = np.array([[1, 2], [4, 5]])
    B = np.array([[7, 8], [9, 10]])
    result = matrix_multiplication(A, B)
    print("Matrix A:")
    print(A)
    print("\nMatrix B:")
    print(B)
    print("\nResult of A * B:")
    print(result)
```

OUTPUT

```
Matrix A:
[[1 2]
 [4 5]]

Matrix B:
[[ 7  8]
 [ 9 10]]

Result of A * B:
[[25 28]
 [73 82]]
```

7. Partial derivates:

```
import sympy as sp
x0,x1=sp.symbols('x0,x1')
f=x0**2+x1**2
partial_derivative_x1=sp.diff(f,x1)
value=partial_derivative_x1.subs({x0:3,x1:4})
print(f"The partial derivative at (x0,x1)=(3,4) is {value}")
```

Output:

```
The partial derivative at (x0,x1)=(3,4) is 8
```

8. Affine and Softmax:

```
import numpy as np
x = np.array([[1, 2, 3, 4]])
weights = np.random.randn(4, 3)
bias = np.zeros((3,))
affine_out = np.dot(x, weights) + bias
print("Output of Affine:", affine_out)
softmax_out = np.exp(affine_out) / np.sum(np.exp(affine_out), axis=1, keepdims=True)
print("Output of Softmax:", softmax_out)
```

Output:

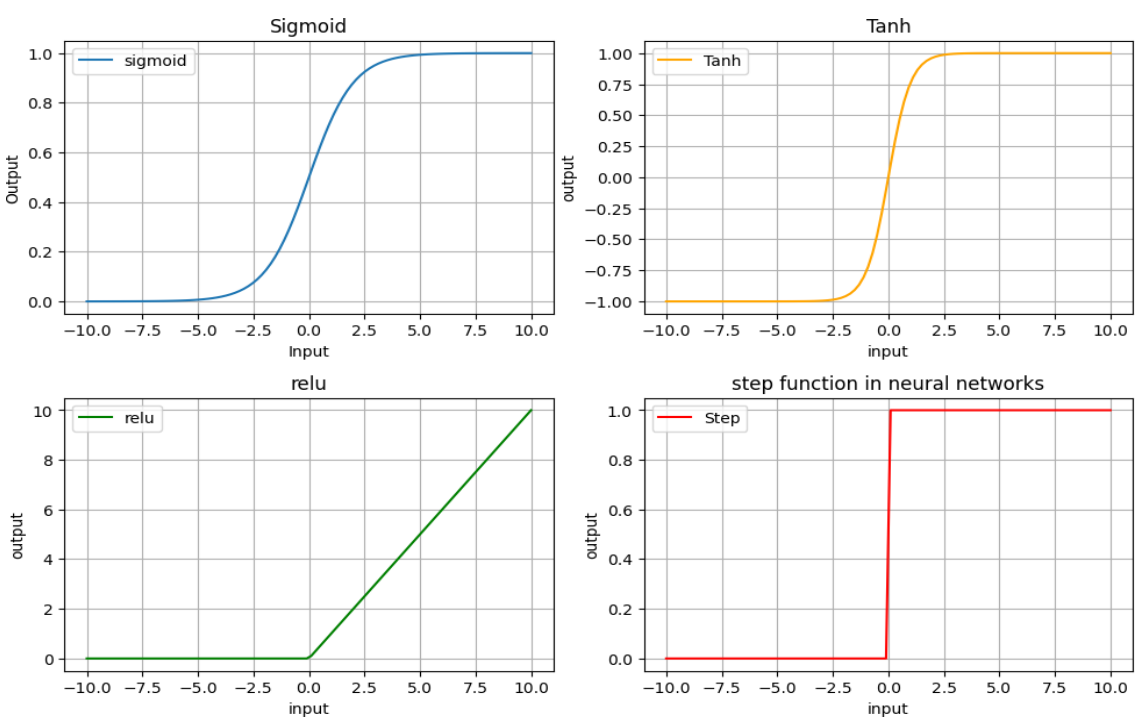
```
Output of Affine: [[3.49952508 4.17225064 1.21132723]]
Output of Softmax: [[0.32668848 0.64016926 0.03314226]]
```

9. Various Activation function:

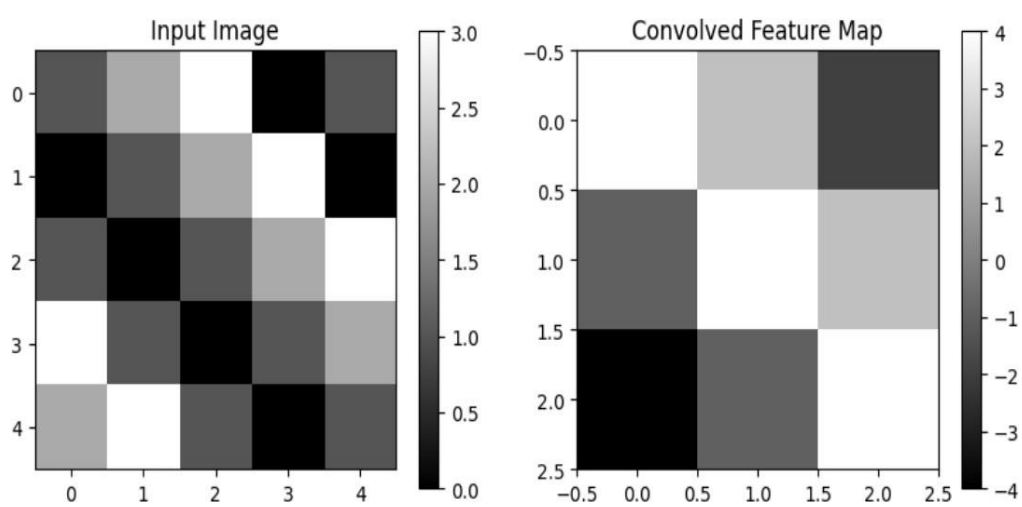
```
import numpy as np
import matplotlib.pyplot as plt

#Sigmoid function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

#Tanh (Hyperbolic Tangent) activation function
def tanh(x):
    return np.tanh(x)

#ReLu
def relu(x):
    return np.maximum(0,x)

#step activation function
def step(x):
    return np.where(x>=0,1,0)

x = np.linspace(-10, 10, 100)  # Define x

plt.figure(figsize=(10,10))

#sigmoid
plt.subplot(3,2,1)
plt.plot(x,sigmoid(x),label='sigmoid')
plt.title('Sigmoid')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
```

```
#tanh
plt.subplot(3,2,2)
plt.plot(x,tanh(x),label='Tanh',color='orange')
plt.title('Tanh')
plt.xlabel('input')
plt.ylabel('output')
plt.grid(True)
plt.legend()

#Relu
plt.subplot(3,2,3)
plt.plot(x,relu(x),label='relu',color='green')
plt.title('relu')
plt.xlabel('input')
plt.ylabel('output')
plt.grid(True)
plt.legend()

#Step
plt.subplot(3,2,4)
plt.plot(x,step(x),label='Step',color='red')
plt.title('step function in neural networks')
plt.xlabel('input')
plt.ylabel('output')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

Output:



10. Convolutional Layer:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve2d

# Generate a simple 2D input (e.g., a 5x5 image)
input_data = np.array([[1, 2, 3, 0, 1],
                       [0, 1, 2, 3, 0],
                       [1, 0, 1, 2, 3],
                       [3, 1, 0, 1, 2],
                       [2, 3, 1, 0, 1]])
# Define a simple 3x3 filter (e.g., edge detection)
filter = np.array([[1, 0, -1],
                   [1, 0, -1],
                   [1, 0, -1]])
# Apply the convolution operation
output = convolve2d(input_data, filter, mode='valid')
# Visualize the input and output
plt.figure(figsize=(10, 4))s
plt.subplot(1, 2, 1)
plt.title("Input Image")
plt.imshow(input_data, cmap='gray')
plt.colorbar()
plt.subplot(1, 2, 2)
plt.title("Convolved Feature Map")
plt.imshow(output, cmap='gray')
plt.colorbar()
plt.show()
```

Output: